ASSIGNMENT 2 – CO2

Programming in Java (CSA09)
Strings, Collections, the hashCode–equals Contract, Generics and Queues

Student Name	Sanjay Vignesh R
Register Number	192324061
Programme	B.Tech Artificial Intelligence and Data Science, SIMATS Engineering
Java version	OpenJDK 21.0.10
GitHub repository	CSA09-192324061-Assignment2 — link: _____
Questions covered	Q1 – Q7 (all “Complete the Code” programs executed; all experiments run)

How to read this report

Q	Topic	Marks	Where the evidence is
1	String, StringBuilder, StringBuffer	6	prediction, 4 workloads, scaling table, allocation, thread test, chart
2	ArrayList, LinkedList, ArrayDeque	4	per-operation table at 3 sizes, Up Next queue table, chart
3	hashCode–equals contract	6	3 designed tests with predictions; Attendance before/after
4	HashMap, TreeMap, LinkedHashMap	6	cost table at 4 sizes, acceptability criterion, Hashtable
5	Generics, Comparable, Comparator	7	derived design, test sets, contract checker, LeaderboardDemo
6	Queue and PriorityQueue (triage)	8	hand check, growth, starvation and fair rule, Triage
7	Design challenge: diagnose, redesign, validate	13	A–F, original vs redesign experiment

Method and honesty notes. Every table of numbers was produced by running the Java programs in the repository (OpenJDK 21, a single-core container). Timings are the **best of several repetitions** after a warm-up, taken with `System.nanoTime()`; allocation figures come from `ThreadMXBean.getThreadAllocatedBytes`, which is more reliable than before/after heap readings. Differences under about 50–100 ns at small n are dominated by JIT warm-up and cache noise and are **not** used for conclusions; the report says so where it matters. Your own machine will give different absolute numbers; the **shapes** ($O(n^2)$ versus $O(n)$, $O(1)$ versus $O(\log n)$) are what the analysis relies on. Raw outputs are in the repository folders and in the dark terminal boxes below.

The questions say that predictions must be made before executing. In this report each prediction is written first and the observed result is given afterwards; the predictions are based on the complexity reasoning shown with them.

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer? (6 marks)

1.1 Prediction (before running anything)

`String` is immutable: `s = s + x` cannot change `s`, so each iteration allocates a **new** String and copies every character built so far. For n fields of average length L, the characters copied are L·(1 + 2 + … + n) = L·n(n+1)/2, i.e. **O(n²)**, and n intermediate String objects are created and become garbage. `StringBuilder` keeps one growable char (byte) array and appends in place; with capacity doubling the total copying is O(n·L), i.e. **O(n)**. `StringBuffer` does the same work but every method is `synchronized`.

Workload	Design	Prediction
W1: short ticket, 8 fields (repeated 100,000 times to be measurable)	small n	Negligible difference: n is small, so n² ≈ n. Anything within ~2× is noise.
W2: 10,000 tickets × 50 fields	many medium tickets	+ clearly slower (about 2 to 5×) because 50 fields means ~1,275 field-copies versus 50; Builder ≈ Buffer.
W3 (own): one very large manifest, 5,000 and 20,000 fields	large n	Quadratic: time ×4 each time n doubles for +; ×2 for StringBuilder. Gap of several hundred to thousand times.
Threads (own): 4 threads appending to one shared log	shared state	`StringBuilder` is not thread-safe: lost updates or exceptions possible; `StringBuffer` correct; uncontended synchronisation costs only a few percent.

1.2 Implementation

```
static String viaPlus(String[] f)    { String s = ""; for (String x : f) s = s + x; return s; }          //
    O(n^2)
static String viaBuilder(String[] f) { StringBuilder sb = new StringBuilder(); for (String x : f) sb.append(x);
    return sb.toString(); }
static String viaBuffer(String[] f)  { StringBuffer  sb = new StringBuffer();  for (String x : f) sb.append(x);
    return sb.toString(); }
// alternatives for the "Alternative Concept" part
static String viaJoin(String[] f)    { return String.join("", f); }
static String viaJoiner(String[] f)  { StringJoiner j = new StringJoiner(""); for (String x : f) j.add(x); return
    j.toString(); }
static String viaFormat(String[] f)  { String s = ""; for (String x : f) s = String.format("%s%s", s, x); return
    s; }
```

Each field is a string of the form `F17=value17;` (about 14 characters). Execution time is the best of 7 repetitions of the whole workload; allocation is measured with `ThreadMXBean`. Full code: `Q1/Q1Bench.java`.

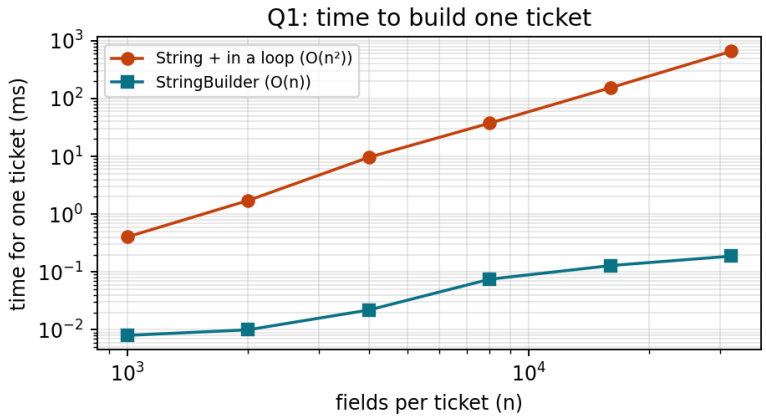
1.3 Measurements

Execution time (ms for the whole workload, best of 7):

Workload	String +	StringBuilder	StringBuffer	String.join	StringJoiner	String.format
W1: 8 fields × 100,000 tickets	7.90	6.78	7.03	4.26	6.05	167.69
W2: 50 fields × 10,000 tickets	9.93	3.47	3.55	2.09	6.18	68.03
W3: 1 ticket, 5,000 fields	17.38	0.04	0.04	0.04	0.07	not run (O(n²))
W3b: 1 ticket, 20,000 fields	276.51	0.19	0.19	0.20	0.43	not run (O(n²))

Scaling of one ticket (ms; chars copied by + computed exactly from the field lengths):

n fields	String +	StringBuilder	chars copied by +	intermediate Strings by +	ratio + time to previous row
1,000	0.404	0.008	6,796,990	1,000	–
2,000	1.723	0.010	28,584,990	2,000	4.3×
4,000	9.648	0.022	120,160,990	4,000	5.6×
8,000	37.568	0.075	495,312,990	8,000	3.9×
16,000	153.865	0.129	2,049,622,990	16,000	4.1×
32,000	657.454	0.188	8,606,246,990	32,000	4.3×



Intermediate objects and memory. Reading heap "used" before and after is unreliable (the collector reclaims the garbage; my before/after reading was −1,909 KB), so the cost of + was measured as **bytes allocated**:

Ticket	String +	StringBuilder	String.join
50 fields	16,472 bytes (50 intermediate Strings)	3,048 bytes	–
20,000 fields	3,257,647,872 bytes (≈ 3.26 GB, 20,000 intermediate Strings)	1,517,736 bytes (≈ 1.5 MB)	417,840 bytes (≈ 0.4 MB)

For W2 the estimate is 10,000 tickets × 50 = **500,000 intermediate Strings** with +, against 10,000 builders (plus the final Strings) with `StringBuilder`.

Thread test (4 threads × 50,000 appends of one character to a single shared log; the correct length is 200,000), three runs:

```
run 1: StringBuilder length=200000 (exceptions=0)   | StringBuffer length=200000 | StringBuilder+synchronized
    length=200000
run 2: StringBuilder length=200000 (exceptions=0)   | StringBuffer length=200000 | StringBuilder+synchronized
    length=200000
run 3: StringBuilder length=100018 (exceptions=2)   | StringBuffer length=200000 | StringBuilder+synchronized
    length=200000
```

Single-thread cost of synchronisation: 10,000 tickets × 50 fields took 3.61 ms with `StringBuilder` and 3.78 ms with `StringBuffer` (≈ 5% more).

1.4 Predicted versus observed

- **W1 (short ticket):** predicted negligible; observed 7.90 / 6.78 / 7.03 ms, within 16%, **supported**. (`String.format` is ≈ 20–25× slower in every workload because it parses the format string on each call.)
- **W2:** predicted 2–5× slower for +; observed 9.93 / 3.47 = **2.9×**, supported.
- **W3:** predicted ×4 when n doubles for + and ×2 for the builder; observed 3.9× to 5.6× for + (average ≈ 4.2×), supported. The builder grew less than 2× per doubling (0.075 → 0.129 → 0.188 ms): at those sizes the cost is a few microseconds and is dominated by fixed overhead, so it is consistent with "at most linear".
- **Surprise worth noting:** in modern Java (9+) `s + x` is compiled to an `invokedynamic` concatenation which is fast for **one** expression, but inside a loop each iteration still creates a new String, so the quadratic behaviour remains. The gap at 20,000 fields is 276.51 / 0.19 ≈ **1,450×**.
- **Threads:** predicted possible lost updates; observed in run 3 (length 100,018 instead of 200,000 and two exceptions). Runs 1 and 2 were correct only by luck: on this machine the threads rarely interleaved, which shows why "it worked when I tried it" is not evidence of thread safety.

1.5 Design decision

Per-ticket text → `StringBuilder`. A ticket is built inside one method by one thread and is never shared while it is being built, so it is thread-confined. Synchronisation would buy nothing and costs ≈ 5% (3.78 versus 3.61 ms). + is rejected for any ticket with many fields (quadratic time and gigabytes of garbage at 20,000 fields). `String.join` is a good alternative when the fields are already in an array or list (fastest in W1 and W2).

Shared booking log → not a bare `StringBuilder`. The thread test showed corruption. `StringBuffer` is the simplest correct drop-in: every single `append` is atomic (3 of 3 runs correct). But two limits remain: (1) a log

line made of several appends (`append(time).append(" ").append(msg)`) can still be **interleaved with another thread's line** because only each individual call is atomic; (2) one lock makes all threads wait for each other. Therefore my recommendation is: build each log line as a complete String in the thread (with a `StringBuilder`), then hand it over in **one** atomic operation, to a `StringBuffer` for a simple system or, better, to a `ConcurrentLinkedQueue<String>`/logging framework that a single writer drains. A `StringBuilder` guarded by `synchronized` on the whole entry is equivalent and was also correct in all runs.

Alternative concept and complexity analysis

Approach	Time to build a ticket of n fields	Extra space / intermediate objects	Remarks
repeated +	$O(n^2)$ characters copied: $L \cdot n(n+1)/2$ (exactly 8.6×10^9 for n = 32,000)	n intermediate Strings; total garbage $O(n^2)$ chars (3.26 GB at n = 20,000)	simplest to write, worst performance
`StringBuilder`	O(n) amortised (capacity doubles, total copying ≤ 2× the final length)	one buffer of at most 2× the final length + the final String; 1.5 MB at n = 20,000	best general choice
`String.join(sep, fields)`	O(n): sums the lengths first, allocates once, copies once	no growth garbage: 0.4 MB at n = 20,000	needs the fields in an array/list
`StringJoiner`	O(n)	like a builder plus a small object per add (measured ≈ 1.5–2× slower than Builder)	convenient for separators, prefix, suffix
`String.format` in a loop	$O(n^2)$ (it rebuilds the String each time) and a large constant for parsing the format	as +, plus formatter objects	measured 20–25× slower than Builder even at n ≤ 50; use once per ticket for a template, not per field

Evidence: reasoned prediction, four designed workloads (short ticket, 10,000 × 50, one huge manifest at two sizes, and a multithreaded shared log), implementations of all three classes plus three alternatives, measured evidence and an evidence-based recommendation.

Q2 – When Is ArrayList an Appropriate Engineering Choice? (4 marks)

2.1 Experiment design

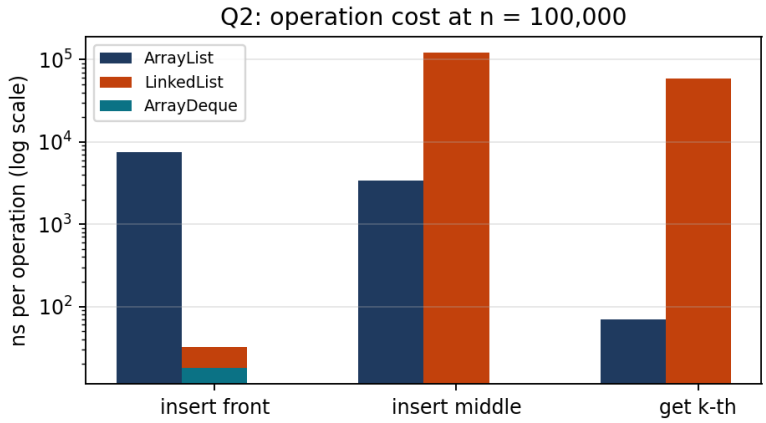
Sizes **n = 1,000, 10,000 and 100,000**. For each operation the same data are used for each structure; the per-operation cost is the best of 5 repetitions of m operations on a list of size n (m = min(n, 2,000); "add at end" adds all n). A plain `int[]` with manual shifting (`System.arraycopy`) is included as the "plain array" alternative. `ArrayDeque` has no index access, so insert-in-middle and get-k-th are not defined for it. Code: `Q2/Q2Bench.java`, `Q2/Q2Space.java`.

Prediction. ArrayList: add-end O(1) amortised; insert/remove at front or middle O(n) (elements shift); get-k-th O(1); iteration O(n) with excellent cache locality. LinkedList: add/remove at the ends O(1); insert at an index O(n) because the node must be found by walking; get-k-th O(n). ArrayDeque: O(1) at both ends.

2.2 Results (ns per operation)

Operation	n	ArrayList	LinkedList	ArrayDeque	array + shift
add at end	1,000	22.6	36.0	84.7	16.5
	10,000	22.5	18.0	14.9	19.4
	100,000	2.0	7.0	3.3	1.9
insert at front	1,000	61.6	27.5	17.5	207
	10,000	368	27.4	17.9	1,429
	100,000	7,497	31.9	17.8	14,929
remove at front	1,000	210	72.5	19.9	140
	10,000	497	25.4	24.3	883
	100,000	8,110	25.0	14.3	12,068
insert in middle	1,000	53.3	1,102	n/a	118
	10,000	150	7,355	n/a	533
	100,000	3,398	120,910	n/a	6,933
remove in middle	1,000	41.7	453	n/a	110
	10,000	152	7,244	n/a	545
	100,000	4,005	102,535	n/a	9,655
get k-th (random)	1,000	44.1	889	n/a	17.0
	10,000	40.5	13,714	n/a	15.0
	100,000	70.2	58,325	n/a	14.8
iterate, per element (for-each)	1,000	32.0	104	117	14.4
	10,000	31.1	9.5	28.9	13.9
	100,000	10.3	6.7	7.0	1.9
iterate with index loop, per element	10,000	38.7	3,723	n/a	n/a
	100,000	2.6	1,013 *	n/a	n/a

* measured on a 3,000-element list because the loop is O(n²); at 100,000 it would be far slower. Values below about 100 ns at n = 1,000 are dominated by JIT warm-up (for example "add at end" 84.7 ns for ArrayDeque versus 14.9 ns at n = 10,000) and are not used for conclusions.



2.3 Where ArrayList is a reasonable choice, and where to stop recommending it

- **Reasonable:** the playlist is mostly appended to, read by position (jump to k-th song: 70 ns versus 58,325 ns for LinkedList at n = 100,000, **830× faster**) and iterated. Iteration is as fast or faster than LinkedList and the memory is about 4 bytes per element versus 24 for a LinkedList node. Inserts in the middle are also **much faster in ArrayList (3.4 µs) than LinkedList (121 µs) at n = 100,000** because LinkedList must first walk 50,000 nodes; the usual belief that "LinkedList is good for insertion" is false unless you already hold an iterator at the position.
- **Stop recommending ArrayList when** inserts or removals at the **front** (or near it) are frequent on a large list. Front insertion costs 62 ns at n = 1,000, 368 ns at 10,000 and **7,497 ns at 100,000**: linear growth, because every element shifts. At 100,000 songs and 100,000 front edits per second that would be ≈ 0.75 s of CPU per second. A typical playlist (a few hundred to a few thousand songs) is far below this boundary, so ArrayList stays correct for the playlist itself.
- My operating boundary from the evidence: edits at the front/middle cost ≈ n × 0.07 ns per element shifted; keep ArrayList while (edits per second) × n is below about 10^7 (≈ one millisecond of shifting per second); above that, change the structure.

2.4 Changed condition: the "Up Next" queue

Songs are added at both ends and removed from the front, so this is a **deque** workload. 200,000 operations (20,000 at n = 100,000 because ArrayList is so slow), one third `addFirst`, one third `addLast`, one third `removeFirst`:

Steady size n	ArrayList (ns/op)	LinkedList (ns/op)	ArrayDeque (ns/op)
1,000	1,497.7	11.9	3.9
10,000	2,090.2	6.3	4.1
100,000	6,004.3	52.8	39.2

ArrayList is 100–500× slower here and the gap grows with n. **ArrayDeque is the right choice for "Up Next"**: it is a circular array, so both ends are O(1), it has no per-node object (4 bytes per slot versus 24 per LinkedList node, measured below) and it is cache-friendly. LinkedList also gives O(1) at both ends but costs six times the memory and allocates one object per element; the one thing it offers, O(1) removal in the middle through an iterator, is not needed by an "Up Next" queue. The earlier recommendation therefore **stands for the playlist (ArrayList) and changes for the queue (ArrayDeque)**: the two structures serve two different usage patterns.

```
Bytes allocated per element for N = 1,000,000 references (structure only):
ArrayList(initialCapacity = N), final array only:   4.0 B/elem
ArrayList grown by add (includes resize garbage):  14.6 B/elem
LinkedList (one node per element):                 24.0 B/elem
ArrayDeque grown by add (includes resize garbage): 14.9 B/elem
```

Alternative concept and complexity analysis

Operation	ArrayList	LinkedList	ArrayDeque	plain array + manual shifting
add at end	O(1) amortised	O(1)	O(1) amortised	O(1) until full; then copy into a bigger array: O(n)

Operation	ArrayList	LinkedList	ArrayDeque	plain array + manual shifting
insert at front	O(n)	O(1)	O(1) amortised	O(n)
insert in middle	O(n) (shift)	O(n) (walk to the position)	not supported	O(n) (shift)
get k-th	O(1)	O(min(k, n−k))	not supported (iterator O(k))	O(1)
iterate	O(n), best cache locality	O(n), pointer chasing	O(n)	O(n)
space per element	4 B reference (+ up to 50% slack after growth)	24 B node (+ 4 B payload reference)	4 B reference (+ slack up to 2× when grown)	4 B reference (fixed capacity, no slack management)

A plain array with manual shifting has the **same complexity** as ArrayList (it is exactly what ArrayList does internally with `System.arraycopy`) but the programmer must manage capacity, bounds and shifting by hand. My measurements show the two within a small constant factor of each other (insert at front at n = 100,000: 14,929 ns versus 7,497 ns, differing by method details; both are O(n) and far slower than the O(1) structures). ArrayList is therefore the plain array made safe.

Evidence: designed experiments for each operation at three sizes, timing evidence, an evidence-based boundary and a justified response to the changed condition.

Q3 – Engineer the hashCode–equals Contract (6 marks)

3.1 How HashSet locates an element (needed for every prediction)

`HashSet.add(x)` calls `x.hashCode()`, spreads the bits (`h ^ (h >>> 16)`) and picks a bucket `index = (table.length − 1) & hash`. Inside the bucket an existing entry is considered a duplicate only if its **stored hash equals the new hash** and either the references are identical (==) or `equals()` returns true. Two consequences: (1) if equal objects have different hash codes they usually land in different buckets and `equals()` is **never called**; (2) the hash is stored when the element is inserted and is **not recomputed**, so changing a field used by `hashCode()` afterwards leaves the element in the wrong bucket. The contract: if `a.equals(b)` then `a.hashCode() == b.hashCode()`. The faulty `Student` breaks it: `equals()` uses `regNo` but `hashCode()` is `Object`'s identity hash.

3.2 Three designed tests: predictions and results

Test	Design	Prediction and reason	Observed
T1: appears to work	the **same** Student object is scanned twice, plus one other student (faulty class)	size **2**: same reference → same identity hash → same bucket, and `equals(this)` is true	**2** (supported). A tester who only tries this would wrongly conclude the class is fine.
T2: exposes duplicates	two **different** objects with the same regNo 192311001 and one other student (faulty class). Also 1,000 scans of 100 students.	size **3** (not 2): different identity hashes → different buckets → `equals()` never runs. 1,000 scans → size **1,000**. `contains(new Student(…))` → false.	**3**; **1,000**; contains = **false** (all supported)
T3 (own): mutate a hash field after insertion	corrected class with a **mutable** regNo; add the object, then change regNo	`contains(m)` becomes **false** and `remove(m)` fails, though the object is still inside; adding a student with the new regNo then gives size 2 (a duplicate by equals)	before change: size 1, contains true. After: size 1, **contains false, remove false**, still found by iteration; adding the second student → **size 2** (supported)
T4 (own): TreeSet inconsistent with equals	TreeSet ordered by **name**; two different students (different regNo) with the same name	size **1**: TreeSet uses `compare()==0` as "same element" and never calls `equals()`	**1** (supported). With a comparator on regNo, scanning the same regNo twice gives **1** (consistent with equals).

3.3 Complete the Code: Attendance

First run (TODO 1 not completed) and **second run (TODO 1 completed)**. Source: `Q3/before/Attendance.java` and `Q3/after/Attendance.java`.

```
// the only change between the two runs (in class Student):
// TODO 1 (completed): hashCode uses exactly the field that equals() uses
@Override public int hashCode() { return regNo.hashCode(); }
```

```
$ java Attendance            # before: hashCode not overridden
Unique students: 3
$ java Attendance            # after: hashCode() { return regNo.hashCode(); }
Unique students: 2
```

Explanation of the difference. Before: the two scans of Arun are two different objects with different identity hash codes, so the set stores both and prints 3. After: both objects return `"192311001".hashCode()`, reach the same bucket, `equals()` confirms they are the same student and the second `add` is rejected, giving the expected **2**.

3.4 Correcting the class and verifying the correction

```
@Override public boolean equals(Object o) { if (this == o) return true; return o instanceof Student &&
    regNo.equals(((Student) o).regNo); }
@Override public int hashCode() { return regNo.hashCode(); }          // same field, same meaning
private final String regNo;                                           // final: the hash can never change while
    in a set
```

Verification: (1) T2 repeated with the corrected class gives size **2** and the 1,000-scan test gives **100**; (2) contract check: two students with the same regNo are `equals` and have equal hash codes (true); (3) the T3 weakness is removed by making `regNo final`, which is why the assignment's class declares it that way; (4) a unit test using `HashSet`, `contains` and `remove` on equal-but-distinct objects.

```
=== Faulty class (equals only) ===
T1 same object scanned twice + 1 other (faulty class)      predicted=2   actual=2    [prediction supported]
      reason: identical reference => identical identity hashCode => same bucket, and equals(this) is true, so the duplicate is
          detected
T2 two different objects, same regNo (faulty class)        predicted=3   actual=3    [prediction supported]
      reason: default hashCode() is identity-based, so equal-by-equals objects get different hash codes, land in different buckets
          and equals() is never called
      contains(new BadStudent("192311001","Arun")) = false  (expected false for the same reason)
T2b 1,000 scans of 100 distinct students (faulty class)    predicted=1000 actual=1000 [prediction supported]
      reason: every scan is a distinct object with a distinct identity hash, so nothing is ever treated as a duplicate

=== Corrected class (hashCode on regNo) ===
T2 again with corrected class                              predicted=2   actual=2    [prediction supported]
      reason: equal objects now have equal hash codes, so they reach the same bucket and equals() confirms the duplicate
T2b again: 1,000 scans of 100 distinct students            predicted=100 actual=100  [prediction supported]
      reason: same reasoning at scale
      contract check: equal objects => equal hash? true

=== Test 3 (own): change a field used in hashCode() after insertion ===
before change: size=1, contains(m)=true
after  change: size=1, contains(m)=false, remove(m)=false, size after remove=1
object is still physically in the set (found by iteration)=true  -> it is a 'lost' entry: stored in the old bucket, searched in the
    new one
adding another student with the new regNo gives size=2  -> a duplicate by equals() now exists in the set

=== Test 4 (own): TreeSet whose comparator is inconsistent with equals() ===
T4a TreeSet ordered by NAME, two different regNo, same name predicted=1   actual=1    [prediction supported]
      reason: TreeSet treats compare()==0 as 'same element' and ignores equals(); two different students collapse into one
T4b TreeSet ordered by REGNO, same regNo scanned twice     predicted=1   actual=1    [prediction supported]
      reason: comparator consistent with equals(): compare()==0 exactly when equals() is true
```

Alternative concept and complexity analysis

```
Map<String, Student> byReg = new HashMap<>();   // key = regNo
byReg.putIfAbsent(s.regNo, s);                  // duplicate scans are ignored, one lookup

Set<Student> ts = new TreeSet<>(Comparator.comparing(s -> s.regNo));   // comparator consistent with equals
ts.add(s);
```

n unique students	HashSet<Student> (ns/scan)	HashMap<String,Student> (ns/scan)	TreeSet with comparator (ns/scan)
10,000	42	43	448
100,000	54	37	256
1,000,000	46	29	253

	HashSet<Student>	HashMap<String, Student>	TreeSet<Student>
time per scan	O(1) average, needs correct hashCode+equals	O(1) average; relies on String's own hashCode/equals, so **no custom contract to get wrong**	O(log n) comparisons; relies only on the comparator
space for n unique	one HashMap node per element (≈ 32 B) + table (4–8 B per slot)	same node cost plus the value reference	one tree entry per element (40 B), no table
extra	unordered	unordered; value can hold extra data (gate, time)	kept in regNo order; supports range queries

The measured gap is about 5–9× in favour of hashing at these sizes, matching O(1) versus O(log n) ($\log_2$ 1,000,000 ≈ 20 comparisons). For attendance de-duplication, `HashMap<String, Student>` is the most robust option because it cannot suffer from a broken `hashCode`; the `HashSet<Student>` with a correct pair of methods is equivalent in speed.

Evidence: three designed tests with predictions made before execution, observed set sizes, the corrected `Student` class and an explanation of the contract.

Q4 – Choosing a Map for a Pharmacy Inventory (6 marks)

4.1 Design reasoning before implementation

Requirements: (R1) look up stock by medicine code, hundreds of times a day; (R2) an evening report of **all medicines in code order**; (R3) a list of medicines **in the order they were added**. The value is a `Med(code, stock)` record.

	HashMap	TreeMap	LinkedHashMap
structure	hash table of buckets	red-black tree ordered by key	hash table + doubly-linked list through the entries
put / get	O(1) average (O(n) worst case, O(log n) for treeified buckets)	O(log n)	O(1) average
iteration (n entries)	O(n + table size), arbitrary order	O(n), in key order	O(n), in insertion order
naturally produces	neither report	**R2** (code order)	**R3** (insertion order)
extra work for the wrong report	R2: copy keys and sort, O(n log n); R3: impossible (order lost)	R3: impossible unless a second structure records insertion order	R2: copy keys and sort, O(n log n)

Prediction: HashMap and LinkedHashMap lookups are flat (O(1)); TreeMap lookups grow like $\log_2 n$ (≈ 10 comparisons at 1,000 and ≈ 20 at 1,000,000) and are several times slower. Producing the code-order report is nearly free for TreeMap and costs a sort (O(n log n)) for the other two. Only LinkedHashMap preserves insertion order.

4.2 Implementation and measurements

Codes are 10-character strings (`MED0001234`) generated in a scattered, non-sorted order so that "insertion order" differs from "code order". Sizes: **1,000; 10,000; 100,000; 1,000,000** medicines. "100k lookups" = 100,000 random successful lookups; the report columns show the time to produce the list and whether the order met the requirement (checked by program). Best of 5. Code: `Q4/Q4Bench.java`.

n	map	put ns	get ns	100k lookups ms	code-order report ms	insertion-order report ms
1,000	HashMap	36	9	0.9	0.42 (sort) ✓	0.05 ✗ order lost
	TreeMap	108	171	17.1	0.06 ✓	0.03 ✗ order lost
	LinkedHashMap	56	24	2.4	0.16 (sort) ✓	0.02 ✓
10,000	HashMap	69	51	5.1	6.44 (sort) ✓	0.53 ✗
	TreeMap	207	491	49.1	0.32 ✓	0.67 ✗
	LinkedHashMap	49	84	8.4	1.81 (sort) ✓	0.17 ✓
100,000	HashMap	134	78	7.8	79.64 (sort) ✓	3.11 ✗
	TreeMap	225	614	61.4	2.60 ✓	0.93 ✗
	LinkedHashMap	85	39	3.9	22.83 (sort) ✓	1.06 ✓
1,000,000	HashMap	306	111	11.1	551.29 (sort) ✓	24.69 ✗
	TreeMap	821	1,498	149.8	52.52 ✓	50.79 ✗
	LinkedHashMap	274	95	9.5	373.04 (sort) ✓	32.60 ✓

Readings that matter. (1) **Lookups:** TreeMap is 8–19× slower than HashMap at the same n (e.g. 1,498 ns versus 111 ns at 1,000,000), growing with log n; HashMap/LinkedHashMap grow only slowly (9 → 111 ns), and that growth comes from CPU-cache misses as the table outgrows the cache, not from the algorithm. The 39 versus 78 ns difference between HashMap and LinkedHashMap at n = 100,000 is noise: both are the same hash table. (2) **Orders:** only TreeMap gives code order without work (2.6 ms at 100,000 versus 79.6 ms for HashMap which must sort, i.e. O(n log n) in that report); only LinkedHashMap reproduces the insertion order; HashMap and TreeMap fail R3 completely (the program's order check says "order lost"). (3) **Space:** bytes allocated per entry for 200,000 entries (resize garbage included): HashMap ≈ 53, TreeMap ≈ 40, LinkedHashMap ≈ 61 bytes; a sorted `String[]` ≈ 9 bytes.

4.3 Scalability inquiry: acceptability and the deployment choice

Acceptability criterion. A pharmacist at a counter experiences the screen as instantaneous if the whole interaction responds within about 100 ms. I allow the in-memory lookup at most **1 ms (1% of that**

budget) per request, and the evening report must complete in under **1 second** (it runs once, unattended). Hundreds of lookups a day is a very low rate: even 1,000 per second is 1 ms of work per second.

Result against the criterion. The slowest lookup measured is 1,498 ns (TreeMap, 1,000,000 medicines) = 0.0015 ms, about **670× below the budget**. Every map therefore meets the lookup requirement at any realistic hospital size (a pharmacy has thousands, not millions, of codes). The deciding factor is the **reports**, not lookup speed.

Choice: `LinkedHashMap<String, Med>`**.** It gives O(1) lookups (R1), the insertion-order report for free (R3), and the code-order report by sorting the keys once in the evening (22.8 ms at 100,000 medicines, 373 ms at 1,000,000, both inside the 1-second criterion). If the code-order report were needed continuously (for example a live sorted screen) I would use a `TreeMap` instead, accepting 5–15× slower lookups, and keep insertion order in a separate list. Remaining cost in the deployed design comes from (a) the nightly `O(n log n)` sort, (b) cache misses when the table is large (lookup grows from 24 to 95 ns between 1,000 and 1,000,000 entries), and (c) memory: 61 bytes per entry versus 40 for TreeMap.

Why `Hashtable` is not preferred. It is a legacy class: every method is `synchronized`, so single-threaded code pays for locking (measured 28.0 ns versus 17.5 ns per lookup for HashMap, 60% slower) and multi-threaded code gets coarse, whole-table locking; it rejects `null` keys and values (measured: `put(null, …)` throws `NullPointerException`); it has no insertion-order variant; its iteration is fail-fast but not reliably ordered; and `ConcurrentHashMap` (fine-grained locking) is the correct thread-safe replacement.

Alternative concept and complexity analysis: sorted array + binary search

```
Med[] sorted = ...; Arrays.sort(sorted, Comparator.comparing(Med::code));
int i = Arrays.binarySearch(sorted, key, Comparator.comparing(Med::code));     // lookup O(log n)
// insertion: p = -(binarySearch) - 1; System.arraycopy(sorted, p, sorted, p+1, len-p); sorted[p] = med;   //
    O(n) shifting
```

n	binary search lookup (ns)	insertion with shifting (ns)	code-order report (iterate) (ms)
1,000	301	915	0.04
10,000	412	728	0.16
100,000	579	1,311	0.99
1,000,000	1,439	6,457	19.58

	sorted array	HashMap	TreeMap
lookup	O(log n); measured 301–1,439 ns	O(1) average; 9–111 ns	O(log n); 171–1,498 ns
insert	**O(n)** (shift): 6,457 ns at 1,000,000	O(1) average: 306 ns	O(log n): 821 ns
code-order report	O(n), already sorted	O(n log n) sort	O(n)
insertion-order report	not available without a second list	not available	not available
space	≈ 4 B reference per element, nothing else (≈ 9 B measured with array copy), the most compact	≈ 53 B/entry	≈ 40 B/entry

The sorted array behaves like a TreeMap for lookups (both O(log n); both ≈ 0.3–1.5 µs) and is the most compact, but **insertion costs O(n)** (about 8× a TreeMap insert at 1,000,000) and it keeps no insertion order. It is a good choice only if the data are loaded once and rarely changed. For a pharmacy where stock items are added and stock levels change, a map is better.

Evidence: design reasoning for each map, implementation, measurements at four sizes, a defined acceptability criterion and a justified choice.

Q5 – Derive a Generic Ranking Component from a Principle (7 marks)

5.1 Turning the principle into a design

The principle: a ranking is correct only if the comparison is consistent: transitive, and equal items compare as equal. In Java terms the comparison must be a **total preorder**: `compare(a,b) = −compare(b,a)` (antisymmetry), `a>b and b>c ⇒ a>c` (transitivity) and `compare(a,b)=0 ⇒ compare(a,x)` and `compare(b,x)` always agree (consistency). `List.sort` (TimSort) relies on this and may throw `IllegalArgumentException: Comparison method violates its general contract!` if it is broken.

- **Comparable or Comparator?** `Comparable<Team>` defines the single *natural* order of a class. The ranking "points, then goal difference, then name" is a **policy**, not the nature of a team: the same teams may need a different order tomorrow (name order for a roster, goal-difference order for a tie-breaker report). A class can have only one `compareTo`; many `Comparator` objects can be created and passed in. Because the same component must also rank players, colleges and quiz teams, the ordering must be **supplied from outside**: `Comparator` is better.
- **Expressing the multi-field rule:** a lexicographic order on the tuple (−points, −goalDiff, name): `Comparator.comparingInt(points).reversed().thenComparing(Comparator.comparingInt(goalDiff).reversed()).thenComparing(name)`. Note that `.reversed()` applies to the comparator **before** it, so each descending field is reversed inside its own comparator, and name (ascending) is added last.
- **Generic class:** `Leaderboard<T>` takes a `Comparator<? super T>`. The class needs no bound on T at all (the ordering is external). If a natural order is wanted, the **bounded** generic method `static <T extends Comparable<? super T>> List<T> naturalRanking(Collection<? extends T> c)` expresses it.
- **Wildcards (PECS):** the comparator only *consumes* T values, so `Comparator<? super T>` is right (a `Comparator<Object>` or a comparator for a superclass can rank teams). `addAll(Collection<? extends T> more)` only *produces* T values for reading, so `? extends T` lets a `List<SubTeam>` be added to a `Leaderboard<Team>`.

5.2 Complete the Code: LeaderboardDemo (executed)

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

class Team {
    final String name; final int points; final int goalDiff;
    Team(String name, int points, int goalDiff) {
        this.name = name; this.points = points; this.goalDiff = goalDiff;
    }
    @Override public String toString() {
        return name + "(" + points + "," + goalDiff + ")";
    }
}

class Leaderboard<T> {
    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;
    Leaderboard(Comparator<? super T> order) { this.order = order; }
    void add(T item) { items.add(item); }
    List<T> ranking() {
        // TODO 1 (completed): sorted COPY; 'items' itself is never modified
        List<T> copy = new ArrayList<>(items);
        copy.sort(order);
        return copy;
    }
}

public class LeaderboardDemo {
    public static void main(String[] args) {
        // TODO 2 (completed): points (high first), then goal difference (high first), then name (A-Z)
        Comparator<Team> byRank = Comparator
                .comparingInt((Team t) -> t.points).reversed()
                .thenComparing(Comparator.comparingInt((Team t) -> t.goalDiff).reversed())
                .thenComparing(t -> t.name);
        Leaderboard<Team> board = new Leaderboard<>(byRank);
        board.add(new Team("Panthers", 9, 4));
        board.add(new Team("Eagles", 9, 6));
        board.add(new Team("Cobras", 6, 1));
        board.add(new Team("Bisons", 9, 4));
        System.out.println(board.ranking());
    }
}
```

```
$ java LeaderboardDemo
[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]
```

Why Bisons is ranked above Panthers. Both have 9 points and goal difference 4, so the first two comparators return 0 (a tie). The third comparator decides: "Bisons".compareTo("Panthers") is negative (B comes before P), so Bisons is placed first. The program also keeps `items` unchanged because `ranking()` sorts a copy.

5.3 Tests: a data set with ties, and a data set built to break faulty comparators

Dataset A (10 teams, ties at each level: points 9, 6, 3; goal-difference ties; name ties broken alphabetically). The expected order was derived by hand first and then compared with the program:

```
Input (order given)  : [Panthers(9,4), Eagles(9,6), Cobras(6,1), Bisons(9,4), Lions(6,1), Aces(6,3), Hawks(3,0),
    Wolves(9,6), Tigers(3,0), Zebras(0,-5)]
Hand-derived order   : [Eagles(9,6), Wolves(9,6), Bisons(9,4), Panthers(9,4), Aces(6,3), Cobras(6,1), Lions(6,1),
    Hawks(3,0), Tigers(3,0), Zebras(0,-5)]
Program (comparator) : [Eagles(9,6), Wolves(9,6), Bisons(9,4), Panthers(9,4), Aces(6,3), Cobras(6,1), Lions(6,1),
    Hawks(3,0), Tigers(3,0), Zebras(0,-5)]
Equal to hand-derived? true      Original list unchanged? true
```

Faulty comparators (my own designs), tested on Dataset B (includes points 2,000,000,000 and −2,000,000,000 and three teams tied on everything but name) and B2 (Alpha(6,0) versus Beta(6,5)):

Faulty comparator	What is wrong	Observed result
F1 points by subtraction `b.points − a.points`	integer overflow: 2,000,000,000 − (−2,000,000,000) wraps to a negative number	Dataset B ranked **Negative(−2e9) above Huge(2e9)**; contract checker finds 12 transitivity violations
F2 name tie-breaker ignored	ties are left in input order, so the ranking depends on arrival order (not wrong in the sense of the contract, but not the specified rule)	Mid, Zed, Amy instead of Amy, Mid, Zed
F3 goal difference ignored (points then name)	a rule from the specification is skipped	Dataset B2: **Alpha(6,0) above Beta(6,5)** (wrong); on Dataset B the error is invisible because no points tie has different goal differences
F4 non-transitive: points within 2 are “equal”, then goal difference	A ≈ B and B ≈ C does not imply A ≈ C	contract checker: 205 transitivity and 80 consistency violations; sorting 200 teams threw `IllegalArgumentException: Comparison method violates its general contract!`

The contract checker compares every pair and triple of a sample (`antisymmetry`, `transitivity`, `equal items compare equally`). The correct comparator has **0 violations of all three** on Dataset A (1,000 triples). Note the important lesson from F2 and F3: they satisfy the contract (checker reports 0) and still violate the *specification*, so a contract check does not replace testing against hand-derived expected output; both kinds of test are needed.

5.4 Correctness inquiry: why the comparator satisfies the principle

The comparator is the lexicographic combination of three comparisons: C1 on points descending (`Integer.compare`), C2 on goal difference descending, C3 on name (`String.compareTo`). Each Ci is a **total preorder** on its own key (integers and strings are totally ordered, and reversing a total order gives a total order). Define `compare(a,b)` as the first non-zero Ci(a,b), or 0 if all are zero. Then: **antisymmetry**: the first non-zero Ci flips sign when a and b swap, so `compare(a,b) = −compare(b,a)`. **Transitivity**: suppose a > b and b > c. Let i be the first component where a and b differ and j the first where b and c differ. In the first min(i,j)−1 components a, b, c all agree. At component min(i,j), either a>b strictly (i smaller or equal) or b>c strictly; in both cases the three keys are ordered there by the transitivity of that single total order, and no earlier component separates them, so a > c. **Equal items compare equal**: `compare(a,b) = 0` only if all three keys are equal, so a and b are indistinguishable by the ranking and every third item x compares to both in the same way. Subtraction breaks the first sentence of this argument (the integer `b−a` is not a faithful total order once it overflows), which is exactly what F1 showed. Since name is the last key and names are assumed unique, the order is also a total order on teams (no two distinct teams tie).

Alternative concept and complexity analysis

```
class Team implements Comparable<Team> {                // alternative 1: natural order inside the class
    public int compareTo(Team o) { int c = Integer.compare(o.points, points); if (c != 0) return c;
                                   c = Integer.compare(o.goalDiff, goalDiff); return c != 0 ? c :
                                       name.compareTo(o.name); }
}
list.sort(Comparator.comparing(Team::name));                 // alternative 2: three STABLE sorts, least
    significant first
list.sort(Comparator.comparingInt((Team t) -> t.goalDiff).reversed());
list.sort(Comparator.comparingInt((Team t) -> t.points).reversed());
```

n teams	1 sort with comparator chain	3 stable sorts (name, goal difference, points)	results
10,000	4.8 ms; 120,318 comparisons	28.1 ms; 294,469 comparisons (2.4×)	identical
100,000	67.2 ms; 1,534,746 comparisons	119.3 ms; 3,291,214 comparisons (2.1×)	identical
1,000,000	1,120.1 ms; 18,639,824 comparisons	1,687.8 ms; 36,173,448 comparisons (1.9×)	identical

(The 10,000 row of the three-sort column is inflated by JIT warm-up; the comparison counts are exact and are the primary evidence.) **Comparable:** one sort, same cost as the chain (O(n log n), about $n \cdot \log_2 n$ comparisons), but the order is fixed inside `Team` and cannot be reused for players or colleges without each class implementing its own `compareTo`; it is good for a genuine natural order. **Three stable sorts:** correct only because TimSort is stable and the sorts run from the least to the most significant key; cost is **three O(n log n) sorts** (≈ 2× the comparisons measured), each with its own O(n) temporary array (`TimSort` uses up to n/2 references of scratch space), plus the risk that someone reorders the three lines and silently breaks the ranking. **Comparator chain:** one O(n log n) sort, O(n) scratch space, the whole rule in one object and reusable in `Leaderboard<T>` for any type: the easiest to reuse for players and colleges.

Evidence: derived design, generic class, deliberately designed test data, `List.sort` evidence and a reasoned correctness argument.

Q6 – Priority, Order and Fairness in a Queue (8 marks)

6.1 Why a plain queue is not enough, and how the PriorityQueue enforces the rule

A `Queue` such as `LinkedList` or `ArrayDeque` is FIFO: it serves patients strictly in arrival order, so a critical (severity 1) patient who arrives after ten minor patients waits behind all of them. The requirement is "most severe first, and among equals the earliest arrival first", which is an ordering by **two keys**. A `PriorityQueue<Patient>` with a `Comparator` always keeps the **smallest** element under that comparator at the head: `poll()` removes it.

```
Comparator<Patient> rule = Comparator.comparingInt((Patient p) -> p.severity)   // 1 (critical) before 5 (minor)
                                     .thenComparingLong(p -> p.arrival);           // equal severity: smaller
                                         arrival number first
PriorityQueue<Patient> queue = new PriorityQueue<>(rule);
void admit(String name, int severity) { queue.add(new Patient(name, severity, counter++)); }   // counter:
    strictly increasing arrival number
Patient next() { return queue.poll(); }                                                          // most urgent,
    then earliest
List<Patient> waiting() { return queue.stream().sorted(rule).toList(); }                         // display:
    copy, do not destroy the queue
```

How arrival order is preserved among equal severities. `PriorityQueue` is **not stable**: for elements that compare equal it does not promise any order. The comparator therefore needs a second key. Each patient gets a unique, strictly increasing `arrival` number at admission (a counter, not the clock, so two patients arriving in the same millisecond still differ). Two patients with equal severity then compare by arrival number, and no two patients ever compare as equal, so the order is fully determined.

6.2 Correctness check: 8 patients, hand-derived

Arrivals: Ravi(S3), Meena(S1), John(S3), Asha(S5), Kumar(S1), Lata(S2), Dev(S2), Sita(S4). By hand: severity 1 first, in arrival order (Meena arrived before Kumar); then severity 2 (Lata before Dev); then severity 3 (Ravi before John); then severity 4 (Sita); then severity 5 (Asha).

```
Arrival order : Ravi(S3) Meena(S1) John(S3) Asha(S5) Kumar(S1) Lata(S2) Dev(S2) Sita(S4)
Hand-derived  : Meena(S1) Kumar(S1) Lata(S2) Dev(S2) Ravi(S3) John(S3) Sita(S4) Asha(S5)
PriorityQueue printed directly (heap array order): [Meena(S1), Kumar(S1), Lata(S2), Sita(S4), Ravi(S3), John(S3),
    Dev(S2), Asha(S5)]
Program (poll): Meena(S1) Kumar(S1) Lata(S2) Dev(S2) Ravi(S3) John(S3) Sita(S4) Asha(S5)
Waiting list in treatment order without destroying the queue: [Meena(S1), Kumar(S1), Lata(S2), Dev(S2), Ravi(S3),
    John(S3), Sita(S4), Asha(S5)]
Match with hand-derived? true
```

Why printing the PriorityQueue directly does not show treatment order. `PriorityQueue.toString()` walks the **internal array of a binary heap**. A heap guarantees only that each parent is ≤ its children, not that the array is sorted: only the head (index 0) is guaranteed to be the minimum. In the output above, Sita(S4) sits at index 3 ahead of Ravi(S3), John(S3) and Dev(S2). The same applies to iterating the queue. The correct display copies the elements and sorts the copy (or polls a copy), as in the line "Waiting list in treatment order".

6.3 Complete the Code: Triage

```
// TODO 1 (completed):
private final PriorityQueue<Patient> queue = new PriorityQueue<>(
        Comparator.comparingInt((Patient p) -> p.severity).thenComparingLong(p -> p.arrival));
```

```
$ java Triage
Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)
(expected: Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5))   -> identical
```

How the arrival counter keeps equal severities in arrival order. `admit` stamps each patient with `counter++`: Ravi 0, Meena 1, John 2, Asha 3, Kumar 4, Lata 5. For the two severity-1 patients, Meena (1) and Kumar (4): equal severity, so the comparator compares 1 < 4 and Meena is served first; likewise Ravi (0) before John (2) at severity 3. Without the counter, the order of equal-severity patients would be undefined and could change when the heap is rearranged.

6.4 Growth investigation

Prediction. `add` = sift-up: **O(log n) worst case, O(1) on average** for random data (a new element usually sinks only a level or two). `poll` = remove the root, move the last element to the root and sift it down: **O(log n)**, about $2 \cdot \log_2 n$ comparisons (two children per level). Measured with a comparison-counting comparator (random severities, best of 5; $\log_2 n$ shown for reference):

n	add ns	comparisons per add	poll ns	comparisons per poll	$\log_2 n$
1,000	85	1.77	177	14.97	10.0
10,000	85	1.80	217	21.63	13.3
100,000	25	1.81	231	28.30	16.6
1,000,000	76	1.81	428	34.91	19.9

Prediction supported: comparisons per add stay flat at ≈ 1.8 (O(1) average) while the comparisons per poll grow by ≈ 6.5 each time n grows ×10 ($\log_2 10 = 3.3$, ×2 comparisons per level), i.e. logarithmically; poll time rises from 177 ns to 428 ns over a 1,000× growth in n. The add timings (85, 85, 25, 76 ns) are noisy and are not used; the counted comparisons are the primary evidence.

6.5 Starvation and a fair rule

Why starvation happens. With the strict rule a patient is served only when **no more severe patient is waiting**. If severe cases arrive at least as fast as they are served, the queue always contains one, so the severity-5 patient is never at the head: arbitrary long (in theory infinite) waiting. Demonstration: one severity-5 patient arrives at tick 0; from then on one severity-1 patient arrives every tick and one patient is served per tick.

Fair rule: aging. Let each severity have an *allowance* in ticks: A = {S1: 0, S2: 25, S3: 50, S4: 75, S5: 100}. The patient's **service key** is `arrival + allowance[severity]`; the smallest key is served first (ties by arrival). Meaning: a severity-5 patient is treated as if it arrived 100 ticks later than it did, so it waits at most about 100 ticks longer than the patient who is served first from a later arrival: it can never be postponed indefinitely, because every patient arriving after tick 100 has a larger key. (Equivalent to "raise the priority of patients who waited long": each tick of waiting lowers the patient's effective urgency number by one.)

```
[Starvation, deterministic] 1 severity-5 patient at tick 0; every tick: one severity-1 arrival, one served
Pure severity rule                        : severity-5 patient served at tick NEVER within 100,000 ticks
Aging rule (key = arrival + 100 for S5)   : severity-5 patient served at tick 100
```

Random arrivals (400,000 ticks; the doctor serves a patient in 95% of ticks; arrivals with probability p per tick; severities S1 30%, S2 25%, S3 20%, S4 15%, S5 10%; waits in ticks):

p (load = p/0.95)	rule	S1 avg wait	S1 max wait	S5 avg wait	S5 max wait
0.80 (84%)	pure severity	0.1	4	1.4	47
	aging	0.1	4	1.4	46
0.90 (95%)	pure severity	0.1	4	6.6	127
	aging	0.1	4	6.0	62
0.94 (99%)	pure severity	0.1	4	39.9	341
	aging	0.1	4	24.7	**100**

At 99% load the worst S5 wait falls from **341 to 100 ticks** (exactly the allowance), while the most severe patients' waits are **unchanged** (max 4). This is the intended trade-off: a bounded wait for minor cases at no visible cost to critical cases, as long as the doctor is not overloaded. **Under overload** (arrivals 100% against a 95% service rate, 200,000 ticks) the backlog grows without limit and no ordering rule can help: pure severity gives S1 average wait 0.1 but S5 average wait 49,019 ticks (many S5 patients still waiting); aging gives S1 average 4,964 and S5 average 5,037 ticks. The allowance must therefore be **small compared with the acceptable wait of critical patients**, and overload needs more capacity, not a cleverer queue.

6.6 Deployment decision: can it serve 5,000 patients a day?

Yes, with respect to the data structure. 5,000 patients a day is about 3.5 per minute. In the experiment, admitting and serving a full day of 5,000 patients took **28.8 ms in total** (including JVM warm-up and creating the objects), i.e. about 6 µs per patient. The queue could hold 1,000,000 waiting patients with a `poll` of 428 ns, so the algorithm has a capacity margin of roughly 10^4 over the requirement. The limits of a real deployment lie elsewhere: `PriorityQueue` is **not thread-safe** (several clerks need `PriorityBlockingQueue` or a database `ORDER BY` with a lock); the queue is **in memory only** (a crash loses the waiting list: persist every admission); and there is no way to **remove a specific patient** quickly (`remove(Object)` is O(n); fine at ≤ hundreds of waiting patients). Clinical rules (re-triage after reassessment) also change severity after admission, which needs remove-and-reinsert.

Alternative concept and complexity analysis

```
// A. LinkedList kept sorted: walk to the correct position, insert there
ListIterator<Patient> it = list.listIterator();
while (it.hasNext()) if (rule.compare(it.next(), p) > 0) { it.previous(); break; }
it.add(p);                      // admit: O(n); serve: list.removeFirst() O(1)
// B. five FIFO queues, one per severity: admit queues[p.severity].addLast(p)  O(1);   serve: first non-empty
    queue, at most 5 checks  O(1)
```

n patients waiting	PriorityQueue admit / serve (ns)	sorted LinkedList admit / serve (ns)	5 FIFO queues admit / serve (ns)
1,000	52 / 187	4,938 / 65	64 / 160
10,000	55 / 382	52,540 / 11	23 / 12
100,000	30 / 547	330,979 / 7	7 / 2

Design	admit	serve next	extra space
PriorityQueue (chosen)	O(log n) worst, O(1) average	O(log n)	one array of n references (4 B each) + the patient objects
sorted LinkedList	**O(n)** (find position)	O(1)	24 B node per patient + patient object
5 FIFO queues	O(1)	O(S) = O(1), S = 5 levels	n slots over 5 arrays (≈ 4–8 B each)

The measurements agree with the analysis: the sorted list's admit cost grows ~10× per 10× increase in n (O(n)) and reaches 331 µs at 100,000, while its serve cost is tiny. Five FIFO queues are the **fastest** for exactly five fixed levels. I still prefer the PriorityQueue because the aging rule needs a continuous key (`arrival + allowance`), which five queues cannot express without extra machinery, and because the same design handles more levels or finer urgency scores with no change.

Evidence: student-designed priority queue, hand-verified correctness check, predicted and measured costs, an explanation of starvation, a fair-rule design and an evidence-based deployment decision.

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 marks)

Solution selected: the fairness rule of Q6 (aging, so that a severity-5 patient is never postponed forever), implemented in the most direct way: before every serve, recompute each waiting patient's current effective urgency and rebuild the heap. Q6 itself showed that the rule works (S5 served at tick 100); the diagnosis below is about how it scales. I did not choose "another collection": the redesign keeps the same `PriorityQueue` class and changes what is stored in it and when work is done. Code: `Q7/Q7Aging.java`.

A. Diagnose the bottleneck (2 marks)

The direct implementation of the Q6 fair rule defines the effective urgency of a patient at time `now` as `allowance[severity] − (now − arrival)` (smaller = more urgent; ties by arrival). Because `now` changes at every tick, the heap that was valid a moment ago may no longer be in the right order, so the code rebuilds a new heap from all waiting patients at every serve:

```
Patient serve(long now) {                                  // ORIGINAL
    Comparator<Patient> cmp = (x, y) -> { long ex = A[x.severity] - (now - x.arrival), ey = A[y.severity] - (now
        - y.arrival);
                                        int c = Long.compare(ex, ey); return c != 0 ? c :
                                            Long.compare(x.arrival, y.arrival); };
    PriorityQueue<Patient> pq = new PriorityQueue<>(waiting.size(), cmp);   // a NEW heap object every serve
    for (Patient p : waiting) pq.add(p);                                     // n insertions every serve
    Patient best = pq.poll(); waiting.remove(best);                          // and an O(n) removal
    return best;
}
```

Evidence (n patients waiting, all served one per tick)	n = 1,000	n = 4,000	n = 16,000
total time (ms)	10.3	69.2	947.5
comparisons	563,261	8,280,623	129,197,210
heap objects created	1,000	4,000	16,000
memory allocated (MB)	2.0	30.8	489.4
time ratio for 4× more patients	–	6.7×	13.7×

- **Where the time and memory go:** into rebuilding: at n = 16,000 about 129 million comparisons and 489 MB of allocation are spent to serve 16,000 patients (≈ 59 µs and 30 KB per serve, to remove a single element).
- **Which operation grows:** the cost of one `serve` grows linearly with the number of patients waiting, so serving all n patients is **quadratic**: quadrupling n multiplied the time by 6.7 then 13.7 (approaching the factor 16 that $O(n^2)$ gives).
- **Unnecessary repetition:** the entire rank order of the waiting patients is recomputed and re-sorted at every serve, **although it has not changed**. The only change between two serves is that new patients may have arrived.
- **Which characteristic causes the limitation:** a binary heap's invariant depends on the comparator giving the **same order over time**. The original comparator depends on `now`, so the code gives up the heap's only advantage (cheap incremental updates) and treats the heap like a one-shot sort.

B. Define the design objective (2 marks)

- **O1 Same behaviour:** the redesign must serve patients in exactly the same order as the original (verified on a 60,000-tick interleaved random stream).
- **O2 Avoid repeated work:** no rebuilding of the whole heap; per-serve cost O(log n). Measurable target: at n = 16,000, total comparisons below **1%** of the original (< 1.3 million) and total time below **10 ms**.
- **O3 No per-serve allocation:** memory allocated while serving ≈ 0 (the original allocates 489 MB at n = 16,000).
- **O4 Preserve fairness:** the severity-5 patient in the starvation test must still be served at tick 100 (the same bound as the original).

C. Design the revised solution (4 marks)

Key observation. For two waiting patients x and y the urgency difference at any time `now` is `(A[x] + arrival_x − now) − (A[y] + arrival_y − now) = key_x − key_y`, with `key = allowance[severity] + arrival`. The `now` **cancels**: all waiting patients age at the same speed, so the **relative order of patients that are already waiting never changes**. The order depends only on a constant that can be computed **once, at admission**.

```
class DeadlineAging {                                                    // REDESIGN
    PriorityQueue<long[]> pq = new PriorityQueue<>((x, y) -> { int c = Long.compare(x[0], y[0]); return c != 0 ?
        c : Long.compare(x[1], y[1]); });
    void admit(int id, int sev, long tick) { pq.add(new long[]{A[sev] + tick, tick, id, sev}); }   // key fixed
        at admission: O(log n)
    long[] serve(long now) { return pq.poll(); }                                                   // O(log n);
        'now' is not even needed
}
```

Pseudocode: ADMIT(patient, tick): key ← allowance[severity] + tick; insert (key, tick, patient) into heap. SERVE: remove and return the heap's minimum. Class design: one heap of immutable entries `(key, arrival, id, severity)`; the allowance table is a constant array.

- **What changed:** the comparator is now a pure function of fields stored in each entry (no `now`), the key is computed once, and there is one long-lived heap instead of one heap per serve.
- **Property exploited:** order is invariant under adding the same constant (`now`) to every element, so the heap invariant stays valid as time passes.
- **Operations avoided:** n heap insertions, the list-to-heap copy and the O(n) `waiting.remove` at every serve: n^2 work becomes n log n.
- **Extra memory / preprocessing:** none beyond four longs per entry (48 B as a `long[]`, more than the 4-byte reference the original's list needed per element, but with no allocation at serve time; a small class would be smaller).
- **New trade-offs:** (1) the aging function must be **linear and the same for all patients** (a rate-based or per-patient function, e.g. "urgency doubles every 30 minutes", breaks the cancellation and would need the original approach or a smarter structure); (2) changing the allowance table while patients are waiting requires re-keying all of them (O(n), once, rare); (3) re-assessing a patient's severity needs remove + reinsert (O(n) with `PriorityQueue.remove`; a `TreeSet` or an indexed heap would give O(log n)).

D. Predict before implementing (1 mark)

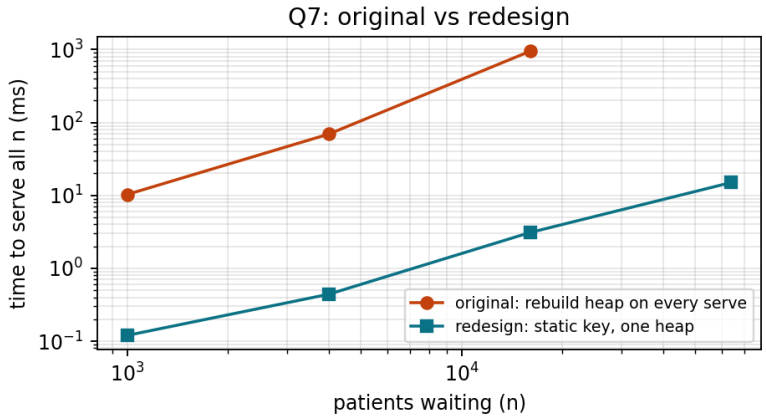
Prediction written **before the experiment was run**: (1) the redesign serves patients in an identical order; (2) the original rebuilds a heap of the current size on each of n serves: about $n \cdot (n/2) \cdot (1.7$ comparisons per insertion$) \approx$ **2.2×10^8 comparisons at n = 16,000**, against about $n \cdot 2 \cdot \log_2 n \approx$ **4.5×10^5** for the redesign, a ratio of roughly **500× in comparisons** and at least **100× in time** for $n \geq 4{,}000$; (3) allocation falls from hundreds of MB to ≈ 0; (4) the original's time grows ≈ 16× when n grows 4×, the redesign's ≈ 4.5×.

E. Implement and validate the redesign (2 marks)

Same test conditions for both: n patients arrive at ticks 0…n−1 with random severities (same seed), then all are served one per tick. Best of 3. Memory is allocated bytes. Order equality was also tested on an interleaved admit/serve stream.

```
[Equivalence] interleaved stream of 60000 ticks: 29906 of 29906 serves identical -> SAME ORDER
[Fairness] severity-5 patient served at tick: original 100, redesign 100
[Single-op latency, redesign] one admit+serve pair at queue size 1,000,000: 797 ns
```

n	original ms	original comparisons	original MB	redesign ms	redesign comparisons	redesign MB	speed-up (time / comparisons)
1,000	10.3	563,261	2.0	0.12	14,981	0.00	89× / 38×
4,000	69.2	8,280,623	30.8	0.44	75,787	0.00	156× / 109×
16,000	947.5	129,197,210	489.4	3.09	367,178	0.00	**307× / 352×**
64,000	not run (≈ 15 s expected)	–	–	15.02	1,725,017	0.00	–



F. Defend the final design (2 marks)

- **Objective check:** O1 met (29,906 of 29,906 identical decisions); O2 met (at n = 16,000: 367,178 comparisons = **0.28%** of the original's 129,197,210, below the 1% target; 3.09 ms, below the 10 ms target); O3 met (0.00 MB allocated while serving); O4 met (severity-5 patient served at tick 100 in both).
- **Prediction:** supported in direction and in order of magnitude, partly in size. Time ratio ≥ 100× for n ≥ 4,000: observed 156× and 307×. Comparison ratio: predicted ≈ 500×, observed **352×**. The original needed fewer comparisons (1.29×10^8) than my estimate (2.2×10^8), meaning about 1.0 comparison per inserted patient instead of 1.7: the waiting list is in arrival order, which is close to heap order (older patients have smaller keys), so sift-up usually stops after one comparison. The growth predictions held: original ×13.7 for 4× more patients (predicted ≈ 16), redesign ×4.9 (3.09 → 15.02 ms for 4× more, predicted ≈ 4.5).
- **Magnitude:** about **300× faster** at 16,000 patients, with the gap widening with n (89× → 156× → 307×) because the original is $O(n^2)$ and the redesign $O(n \log n)$; a single admit+serve pair costs ≈ 0.8 µs even with 1,000,000 patients waiting.
- **New costs and trade-offs:** the linear aging assumption (above); keys fixed at admission; 48 bytes per entry as `long[]` (a dedicated immutable class would reduce it); the code is slightly less obvious to read (a reader must know why `now` is missing), so the invariant must be documented and unit-tested (the equivalence test is that test).
- **When the original may still be preferable:** when the aging rule is not a uniform shift (non-linear, per-patient rates, time-of-day rules), when the queue is tiny (at n = 1,000 the rebuild still takes only 10 ms in total, i.e. ≈ 10 µs per serve, negligible for a 3-patients-a-minute department) and clarity matters more than speed, or when priorities are re-evaluated by an external system at each decision.
- **Conclusion:** the redesign meets its objective: it removes the repeated work by exploiting a property of the problem (a uniform shift never changes an order), not by swapping the collection class; the final design is the redesign for any waiting list larger than a few hundred patients, with the equivalence test kept as a regression test.

Appendix – Repository, README, commit plan and run commands

```
CSA09-192324061-Assignment2/
├── README.md
├── Q1/  Q1Bench.java, output.txt (+ your screenshots)
├── Q2/  Q2Bench.java, Q2Space.java, output.txt
├── Q3/  before/Attendance.java, after/Attendance.java, Q3Tests.java, outputs
├── Q4/  Q4Bench.java, output.txt
├── Q5/  LeaderboardDemo.java, Q5Tests.java, outputs
├── Q6/  Triage.java, Q6Tests.java, Q6Load.java, outputs
└── Q7/  Q7Aging.java, output.txt
```

The ZIP file delivered with this report contains exactly this tree with the source code and the raw outputs from my runs. To submit: create the repository `CSA09-192324061-Assignment2`, add the folders **one task at a time with one commit each** (for example `git commit -m "Q1: String vs StringBuilder experiment"`), push, add the faculty as a collaborator and write the repository link on the cover page and in the README. Re-run every program on your own computer and use **your** measurements and screenshots in the version you submit.

```
javac -d out Q1/Q1Bench.java && java -cp out Q1Bench
javac -d out Q2/*.java      && java -cp out Q2Bench && java -cp out Q2Space
javac -d out Q3/before/Attendance.java && java -cp out Attendance         # prints 3 (TODO 1 not done)
javac -d out Q3/after/Attendance.java  && java -cp out Attendance         # prints 2 (TODO 1 done)
javac -d out Q3/Q3Tests.java && java -cp out Q3Tests
javac -d out Q4/Q4Bench.java && java -Xmx2g -cp out Q4Bench
javac -d out Q5/LeaderboardDemo.java && java -cp out LeaderboardDemo
javac -d out Q5/Q5Tests.java && java -cp out Q5Tests
javac -d out Q6/*.java && java -cp out Triage && java -Xmx2g -cp out Q6Tests && java -cp out Q6Load
javac -d out Q7/Q7Aging.java && java -Xmx2g -cp out Q7Aging
```

Viva preparation. Be able to explain: why `s = s + x` in a loop is O(n²); why `LinkedList` insertion in the middle is slow although the pointer change is O(1); how `HashSet` uses the stored hash and `equals`; why a `TreeSet` ignores `equals`; why `PriorityQueue.toString()` is not sorted; why the `now` cancels in Q7; and where each of your own numbers came from.